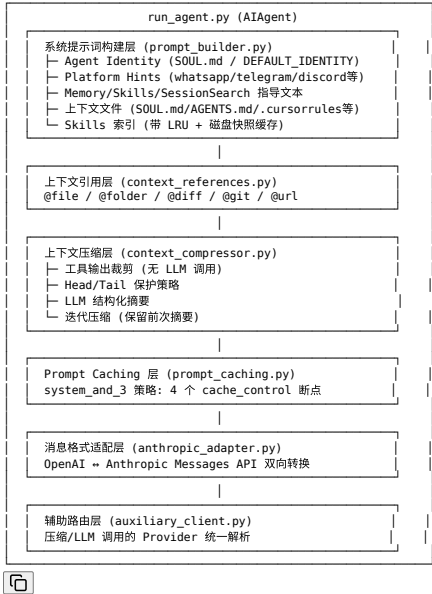


上下文管理

一、总体架构概述

Hermes Agent 的上下文管理方案采用了**分层解耦**的设计思路，将上下文管理拆分为多个职责明确的独立模块：



二、系统提示词构建层 (prompt_builder.py)

2.1 分层结构

系统提示词由多个独立片段组合而成，顺序如下：

优先级	组成部分	来源	稳定性
1	Agent Identity	SOUL.md 或 DEFAULT_AGENT_IDENTITY	静态 (会话内不变)
2	Platform Hints	PLATFORM_HINTS dict (whatsapp/telegram等)	静态
3	Memory/Skills 指导	MEMORY_GUIDANCE / SKILLS_GUIDANCE 等	静态
4	项目上下文文件	SOUL.md / AGENTS.md / .cursorsrules	静态 (按需扫描)
5	Skills 索引	~/hermes/skills/ 目录扫描	静态 (缓存)

2.2 上下文文件优先级系统

build_context_files_prompt() 实现了**互斥优先级**机制——同一时刻只加载一种项目上下文文件：

```
project_context = (
    _load_hermes_md(cwd_path) # 优先级1: .hermes.md / HERMES.md
    or _load_agents_md(cwd_path) # 优先级2: AGENTS.md
    or _load_claude_md(cwd_path) # 优先级3: CLAUDE.md
    or _load_cursorsrules(cwd_path) # 优先级4: .cursorsrules
)
```

2.3 安全注入：提示词威胁扫描

上下文文件在注入系统提示词之前，会经过 _scan_context_content() 检测：

```
_CONTEXT_THREAT_PATTERNS = [
    (r'ignore\s+(previous|all|above|prior)\s+instructions', "prompt_injection"),
    (r'do\s+not\s+tell\s+the\s+user', "deception_hide"),
    ...
]
```

以及零宽字符检测 (\u200b 等不可见 Unicode)，这些字符是提示词注入的常用手法。

Note

防御性设计：
信任但验证： Hermes 对上下文文件采用“加载但扫描”策略，而非完全拒绝。这在灵活性（允许用户自定义项目规则）和安全性（阻止明显注入）之间取得了平衡。扫描到的威胁被记录日志并替换为占位符 [BLOCKED: ...]，而不是直接崩溃。

这与 MCP 的安全哲学形成对比：MCP 默认拒绝，Hermes 默认接受但验证。在实际使用中，这种差异取决于用户场景——开发者工具需要灵活性，生产环境可能需要更严格的隔离。

2.4 Skills 索引的二级缓存

build_skills_system_prompt() 实现了**进程内 LRU + 磁盘快照**的两级缓存：

缓存层级	持久性	失效机制
进程内 LRU	无	skills 文件 mtime/size 变化
磁盘快照	跨进程	manifest (mtime+size) 不匹配
文件系统扫描	每次读取	无 (每次重新解析)

缓存 Key 包含 (skills_dir, external_dirs, available_tools, available_toolsets)，确保工具集变化时缓存失效。

Note

缓存失效粒度的艺术：很多系统的缓存失效逻辑过于粗糙（简单 TTL）或过于复杂（依赖注入）。Hermes 使用 mtime + size 的 manifest 方式，既能在文件修改时自动失效，又避免了每次都解析文件内容。这是一种务实的工程选择。

同时值得注意的是，external_dirs（外部依赖目录）没有快照缓存，只能每次扫描。这意味着外部目录越大、文件越多，冷启动性能会线性下降。

三、上下文引用系统 (context_references.py)

3.1 @-引用语法设计

Hermes 支持在消息中嵌入 @ 引用，格式如下：

```
@file:path/to/file          # 文件内容
@file:path/to/file:10-20    # 文件指定行范围
@folder:path/to/dir         # 目录结构
@diff                       # git diff
@staged                     # git diff --staged
@git:N                      # git log -N -p
@url:https://...           # 网页内容
```

3.2 安全边界控制

preprocess_context_references() 实现了**双重沙箱**：

- 路径边界**：allowed_root 参数将所有相对路径解析限制在指定目录内，防止 ../ 逃逸。
- 敏感路径拦截**：_ensure_reference_path_allowed() 明确拦截：
 - ~/.ssh/* (私钥, authorized_keys)
 - ~/hermes/.env (凭证)
 - 其他 30+ 敏感文件模式

```
blocked_exact = {home / rel for rel in _SENSITIVE_HOME_FILES}
blocked_dirs = [home / rel for rel in _SENSITIVE_HOME_DIRS]
```

3.3 Token 预算控制

引用内容注入有严格的 Token 限制：

- 硬限制**：注入内容不得超过上下文窗口的 50%（直接拒绝）
- 软限制**：超过 25% 时发出警告但不阻止

```
injected_tokens > 50% context_length → BLOCK (拒绝注入)
injected_tokens > 25% context_length → WARN (记录警告)
```

Note

Token 预算作为“最后一公里”安全阀：即使所有路径安全检查都通过，Token 预算控制确保了大文件引用不会“淹没”对话上下文。这是一种“纵深防御”（Defense in Depth）——每层检查都有可能被绕过，但多层检查叠加使绕过变得几乎不可能。

25%/50% 的阈值是经验值：在实践中，引用内容超过上下文窗口四分之一时，对话质量已经开始下降，因为模型需要“回忆”的内容比例增高。

四、上下文压缩系统（context_compressor.py）

4.1 压缩触发机制

压缩系统采用**主动预检** + **被动阈值**的双重触发：

```
# 主动预检: API 调用前使用粗糙估算
def should_compress_preflight(self, messages):
    rough_estimate = estimate_messages_tokens_rough(messages)
    return rough_estimate >= self.threshold_tokens
```

```
# 被动阈值: API 响应后使用精确 Token 数
def should_compress(self, prompt_tokens=None):
    tokens = prompt_tokens or self.last_prompt_tokens
    return tokens >= self.threshold_tokens
```

触发阈值默认是上下文窗口的 50%（可配置）。

4.2 压缩算法：四阶段流水线

原始消息列表

- ▼ [阶段1: 工具输出裁剪]
- 无 LLM 调用，直接将 >200 chars 的旧工具结果替换为占位符
- ▼ [阶段2: Head/Tail 边界确定]
- Head: 前 N 条消息 (默认 3 条) 完整保留
- Tail: 基于 token budget 的动态保护窗口 (默认 ~20K tokens)
- ▼ [阶段3: LLM 结构化摘要]
- 对 middle turns 调用 LLM 生成结构化摘要
- 摘要因迭代压缩而保留前次结果
- ▼ [阶段4: 工具对完整性修复]
- 清理 orphan tool_call / tool_result 对
- (压缩可能破坏 tool_use → tool_result 的对应关系)
- ▼ 输出

Note

个人感觉现有的处理链路存在逻辑缺陷，工具输出的截断（阶段 1）不应阻塞知识抽取（阶段 3）。上下文裁剪和记忆压缩应该走并行的处理路线：裁剪仅对当前 Prompt 的组装生效，而摘要和记忆模块的清洗对象必须是无损的原始上下文快照。一旦工具返回结果在前端被强行截断，摘要链路拿到的就是缺乏语义密度的残缺数据，极易导致隐性知识挖掘失败

4.3 结构化摘要模板

摘要采用固定的 6 字段结构：

```
## Goal
[用户目标]

## Constraints & Preferences
[用户偏好、编码风格、约束、重要决策]

## Progress
### Done   [已完成工作]
### In Progress [进行中]
### Blocked [阻塞项]

## Key Decisions
[重要技术决策及原因]

## Relevant Files
[已读/修改/创建的文件]

## Next Steps
[下一步计划]

## Critical Context
[关键值、错误消息、配置等]
```

PS：以下观点不敢保证完全正确性

Note

1. 压缩和截断造成的信息丢失问题

压缩和截断本质上都是信息丢失（永久性），为了解决这个问题建议可以利用本地文件系统，采用读写解耦的**分级记忆（Tiered Memory）**架构：主干路由执行激进裁剪以控制 Token 预算，旁路异步持久化无损的全量交互轨迹（含完整工具输出）。在此机制下，通过 System Prompt 注入强约束触发器：当结构化摘要（缓存）细节不足以支撑推理时，强制 Agent 依据索引定向检索本地归档（主存），实现高保真数据的按需召回，降低幻觉存在的情况

[Memory Access Directive | 记忆检索指令]

当前上下文为经过有损压缩的结构化摘要，仅包含高层逻辑与状态快照。完整的历史轨迹（含全量对话及未经裁剪的工具调用结果）已持久化归档至本地：[FILE_PATH]。

Action Rule: > 当你在当前摘要中无法获取足够支撑推理的细节，或需要查证历史报错堆栈、长文本参数等隐性知识时，严禁产生幻觉或自行盲目推断。你必须优先调用 [Search_Tool_Name] 工具，通过精准关键词在上述归档文件中进行检索，召回原始上下文后再继续执行当前任务。



2. 已读文件的问题

压缩时若丢弃实际内容而保留“已读”状态，会形成**悬空引用**并引发模型幻觉。消除这种“状态与数据不一致”的路径的方法：

- 按需重载**：保留状态作为索引，在需要时将实际内容重新拉回上下文。
- 级联擦除**：确保所见即所得，内容一旦被弃，同步删除对应的“已读”状态。

4.4 工具对完整性修复

压缩后可能出现两种异常：

- Orphan tool result**：工具结果存在，但调用它的 assistant 消息已被压缩 → 删除该工具结果
- Orphan tool call**：assistant 消息有 tool_calls，但对应的工具结果被截断了 → 插入占位符 [Result from earlier conversation – see context summary above]

```
# 检测 orphan tool result
orphaned_results = result_call_ids - surviving_call_ids
messages = [m for m in messages
             if not (m.get("role") == "tool" and m.get("tool_call_id") in orphaned_results)]

# 添加 stub tool result
missing_results = surviving_call_ids - result_call_ids
for tc in msg.get("tool_calls"):
    if cid in missing_results:
        patched.append({
            "role": "tool",
            "content": "[Result from earlier conversation – see context summary above]",
            "tool_call_id": cid,
        })
```



4.5 Head/Tail 对齐修复

压缩边界可能落在工具对的中间位置：

- Tail 对齐** (_align_boundary_backward)：边界前的连续 tool results 被一并纳入压缩区，避免中间截断
- Head 对齐** (_align_boundary_forward)：压缩起始位置如果是 tool result，向后滑动到非 tool 消息

Note

消息格式完整性是 API 正确性的前提：Anthropic 的 Messages API 对工具调用的格式有严格要求——每个 tool_use 必须有匹配的 tool_result，每个 tool_result 必须对应一个 tool_use。压缩操作如果破坏了这种对应关系，会导致 API 返回 400 错误。

很疑惑的地方在于为什么会出现不完整的情况，常规的上下文管理都会做完整性校验，即使触发压缩保护，也是基于“对话块”进行原子级隔离，确保 Tool-Call 和 Result 成对闭环

五、Prompt Caching (prompt_caching.py)

5.1 system_and_3 策略

Anthropic 的 Prompt Caching 支持 cache_control 标记，可以将已处理的内容缓存在服务器端，**减少 75% 的输入 token 费用**。

```
# 缓存 4 个断点: system prompt + 最后 3 条非系统消息
breakpoints_used = 0

if messages[0].get("role") == "system":
    _apply_cache_marker(messages[0], marker)
    breakpoints_used += 1

remaining = 4 - breakpoints_used # 剩余可用断点数
non_sys = [i for i in range(len(messages)) if messages[i].get("role") != "system"]
for idx in non_sys[-remaining:]: # 从后往前选
    _apply_cache_marker(messages[idx], marker)
```



缓存 TTL 支持 5m（默认）和 1h。

Note

PS: TTL 采用**滑动过期**（Sliding Expiration）机制，自最后一次活跃时间（Last Active Time）起算。任何新的 API 调用均会触发状态刷新，使计时器清零重置

5.2 缓存断点选择逻辑

```
[System (cache)] [msg1] [msg2] ... [msgN-3] [msgN-2 (cache)] [msgN-1 (cache)] [msgN (cache)]
      ↑                ↑
      被压缩区（不在缓存中）        最后3条非系统消息进入缓存
```



选择逻辑：**系统消息固定缓存 + 从消息列表末尾选取剩余断点**。

Note

动态内容 vs 静态内容的平衡：Anthropic 的缓存策略意味着越近的消息越不会被缓存（因为缓存点在固定位置）。在 Hermes 的设计中，“最后 3 条非系统消息”可能包含用户刚输入的内容，这些内容反而没有被缓存。

这可能是因为某些异步 agent（例如 CC 的记忆和压缩 agent）会对主 agent 进行 fork，然后通过用户块给出任务提示词，这种缓存策略能让主 agent 和旁路的其他 agent 都有大部分内容能命中缓存

六、消息格式适配 (anthropic_adapter.py)

6.1 OpenAI ↔ Anthropic 双向转换

convert_messages_to_anthropic() 负责将 OpenAI 格式消息转换为 Anthropic Messages API 格式，核心转换包括：

OpenAI 格式	Anthropic 格式
{ "role": "system", "content": "..." }	提取为独立的 system 参数
{ "role": "assistant", "content": "...", "tool_calls": [...] }	转换为 text + tool_use blocks
{ "role": "tool", "tool_call_id": "xxx", "content": "..." }	转换为 tool_result block
{ "role": "user", "content": [...] } (多模态)	递归转换每个 content block
连续相同角色消息	合并 (user) 或追加 (assistant)

6.2 OAuth 兼容性处理

当使用 Claude Code OAuth token 时，build_anthropic_kwargs() 会执行额外的兼容性转换：

- 系统前缀替换**：Hermes Agent → Claude Code
- 工具名前缀**：tool_name → mcp_tool_name
- 身份伪装**：发送 Claude Code 的 User-Agent 和版本号

```
if is_oauth:
    block["text"] = block["text"].replace("Hermes Agent", "Claude Code")
    tool["name"] = "mcp_" + tool["name"]
```



Note

OAuth 适配是提供商绑定的典型案例：Claude Code 的 OAuth 基础设施 (platform.claude.com) 要求请求带有特定的 user-agent 和 x-app 头部，否则返回 500 错误。这种“协议级绑定”使得非官方客户端（如 Hermes）需要模拟 Claude Code 的行为才能正常使用 OAuth，

这体现了 API 生态的一个现实问题：当提供商（如 Anthropic）的 OAuth 服务与特定客户端（如 Claude Code）强绑定时，第三方客户端要么放弃 OAuth（使用普通 API key），要么精确模拟客户端行为（脆弱但可用）。

6.3 严格角色交替强制

Anthropic Messages API 要求消息角色严格交替 (user → assistant → user → ...)。convert_messages_to_anthropic() 实现了自动合并：

```
# 连续 assistant 消息 → 合并 content blocks
if fixed[-1]["role"] == m["role"] == "assistant":
```

```
fixed[-1]["content"] = prev_blocks + curr_blocks

# 连续 user 消息 → 合并内容文本
if fixed[-1]["role"] == m["role"] == "user":
    fixed[-1]["content"] = prev_content + "\n" + curr_content
```

七、辅助路由层（auxiliary_client.py）

7.1 Provider 统一解析

auxiliary_client.py 实现了 **单一入口**的 **Provider 解析链**，所有辅助任务（压缩、web 提取、视觉分析等）都通过 call_llm() / async_call_llm() 调用，解析顺序：

文本任务自动检测链：

1. OpenRouter（OPENROUTER_API_KEY）
2. Nous Portal（~/.hermes/auth.json）
3. Custom 端点（config.yaml base_url + OPENAI_API_KEY）
4. Codex OAuth（Responses API → chat.completions adapter）
5. 原生 Anthropic
6. Direct API-key providers（z.ai/GLM, Kimi, MiniMax）

视觉任务自动检测链：

1. 主 Provider（如果是支持的视觉后端）
2. OpenRouter → Nous → Codex → Anthropic → Custom

7.2 Codex Responses API 适配器

Codex 的 OAuth token 只能访问 Responses API（而非 chat.completions）。_CodexCompletionsAdapter 实现了透明的协议转换：

```
# 输入: chat.completions 格式 kwargs
{
    "model": "gpt-5.2-codex",
    "messages": [...],
    "tools": [...]
}

# 转换为 Responses API 格式
{
    "model": "gpt-5.2-codex",
    "instructions": "You are a helpful assistant.",
    "input": [{"role": "user", "content": [...]}],
    "tools": [...],
    "store": False,
}
```

输出: 转换回 chat.completions 格式

7.3 异步事件循环隔离

Async 客户端的缓存 Key 包含 loop_id，确保每个事件循环有独立的客户端实例：

```
cache_key = (provider, async_mode, base_url or "", api_key or "", loop_id)
```

这避免了跨事件循环复用 AsyncOpenAI 客户端导致的死锁。

Note

事件循环亲和性是异步 SDK 的隐形陷阱：httpx.AsyncClient 绑定到创建时的事件循环，但在后台任务（run_in_background）中，事件循环可能已经切换。Hermes 通过将 loop_id 纳入缓存键来确保每个循环获得独立的客户端实例。

这种问题在短生命周期脚本中不会出现，但在 gateway 模式（长期运行、多个并发请求）下会成为稳定性杀手。

八、安全层（redact.py）

8.1 多层脱敏策略

redact_sensitive_text() 实现了 8 种敏感信息检测模式，按执行顺序：

```
# — 1. 已知前缀 API Key (23 种 Provider) —————
# 词边界环视 (?<!/) 防止误匹配普通文本中的子串
_PREFIX_PATTERNS = [
    r'sk-[A-Za-z0-9_-]{10,}',           # OpenAI / Anthropic / OpenRouter
    r'ghp_[A-Za-z0-9]{10,}',           # GitHub PAT (经典)
    r'github_pat_[A-Za-z0-9]{10,}',    # GitHub PAT (细粒度)
    r'xox[baprs]-[A-Za-z0-9]{10,}',    # Slack tokens
    r'AiZa[A-Za-z0-9_-]{30,}',         # Google API keys
    r'AKIA[A-Z0-9]{16}',               # AWS Access Key ID (固定长度)
    r'sk_live_[A-Za-z0-9]{10,}',       # Stripe Live key
    r'sk_test_[A-Za-z0-9]{10,}',       # Stripe test key
    r'SG\.[A-Za-z0-9_-]{10,}',         # SendGrid
    r'hf_[A-Za-z0-9]{10,}',            # HuggingFace
    r'tvly-[A-Za-z0-9]{10,}',          # Tavily
    ... # 共 23 种, 含 Perplexity / Fal.ai / Replicate / npm / PyPI / DigitalOcean 等
]
_PREFIX_RE = re.compile(
    r'(?<![A-Za-z0-9_-])( " + "|"'.join(_PREFIX_PATTERNS) + r" " )?(?![A-Za-z0-9_-])"
)

# — 2. 环境变量赋值 KEY=value (含引号) —————
# 匹配名含 API_?KEY|TOKEN|SECRET|PASSWORD|CREDENTIAL|AUTH 的所有变量, 不区分大小写
_SECRET_ENV_NAMES = r'(?:(?:API_?KEY|TOKEN|SECRET|PASSWORD|PASSWD|CREDENTIAL|AUTH))'
_ENV_ASSIGN_RE = re.compile(
    rf'([A-Z_])*{_SECRET_ENV_NAMES}[A-Z_]*\s*=\s*([\'"])?([S+])\2"',
    re.IGNORECASE,
)

# — 3. JSON 字段 "apiKey": "value" —————
# 覆盖 apiKey / token / password / secret / access_token 等 12 种常见字段名
_JSON_KEY_NAMES = r'(?:(?:api_[Kk]ey|token|secret|password|access_token|'
refresh_token|auth_token|bearer|secret_value|raw_secret|secret_input|key_material)"
_JSON_FIELD_RE = re.compile(rf'({_JSON_KEY_NAMES})\s*:\s*"([~"]+)"', re.IGNORECASE)

# — 4. Authorization 请求头 —————
_AUTH_HEADER_RE = re.compile(r'(Authorization:\s+Bearer\s+)([S+)*', re.IGNORECASE)

# — 5. Telegram Bot Token (两种格式均支持) —————
bot123456789:ABCDEF... 或 123456789:ABCDEF...
_TELEGRAM_RE = re.compile(r'(bot)?(\d{8,}):([A-Za-z0-9]{30,})')

# — 6. PEM 私钥块 (RSA / EC / OPENSSH 等所有 PKCS#8 变体) —————
_PRIVATE_KEY_RE = re.compile(
    r'-----BEGIN[A-Z_]*PRIVATE KEY-----([S\S])*?-----END[A-Z_]*PRIVATE KEY-----'
)

# — 7. 数据库连接字符串 (protocol://user:PASSWORD@host) —————
# 支持 postgres(postgresql) / mysql / mongodb(+srv) / redis / amqp
_DB_CONSTR_RE = re.compile(
    rf'((?:postgres(?:ql)?|mysql|mongodb(?:\+srv)?|redis|amqp)://[^\:]+\:[^@]+\{[@]}\{@"',
    re.IGNORECASE,
)

# — 8. E.164 国际电话号码 (Signal / WhatsApp 等) —————
# 负向前瞻 (?![A-Za-z0-9]) 防止将十六进制字符串误识为电话号码
_SIGNAL_PHONE_RE = re.compile(r'^(+[1-9])d{6,14})(?!([A-Za-z0-9])")')
```

8.2 Token 保留策略

```
短 Token (<18 字符): 完全遮蔽 → ***
长 Token: 保留前 6 + 后 4 → sk-abcde...ghij

def mask_token(token: str) -> str:
    if len(token) < 18:
        return "****"
    return f"[token[:6]]...{token[-4:]}"
```



Note

脱敏的完整性决定安全下限：正则表达式脱敏的核心挑战是**覆盖率 vs 误报率**的权衡。Hermes 选择了相对保守的模式（如 sk-[A-Za-z0-9_-]{10,} 要求至少 10 个字符），这意味着它不会误遮蔽普通的 sk 开头字符串，但可能遗漏非标准格式的密钥。

最关键的盲点：redact.py 是运行时脱敏（应用于日志输出），而非编译时或传输时脱敏。这意味着如果 API 请求本身被拦截，脱敏不会保护任何东西。正确使用场景是：日志文件泄露、打印输出意外暴露、调试信息传播等"事后泄露"场景。

九、Token 预算管理 (model_metadata.py)

9.1 模型上下文长度解析链

get_model_context_length() 实现了 10 级回退解析：

- 0. 显式 config 覆盖（用户明确设置）
 - ❌ 不满足
- 1. 持久化缓存（磁盘 yaml, model@base_url 作为 key）
 - ❌ 不满足
- 2. 端点 /models 元数据（仅限非知名 provider 的自定义端点）
 - ❌ 不满足
- 3. 本地服务器查询 (Ollama/LM Studio/vLLM/llama.cpp)
 - ❌ 不满足
- 4. Anthropic /v1/models API（需要 sk-ant-api*, 不支持 OAuth）
 - ❌ 不满足
- 5. Provider-aware models.dev 查询
 - ❌ 不满足
- 6. OpenRouter 实时 API 元数据
 - ❌ 不满足
- 7. 模糊硬编码默认值（按模型家族匹配）
 - ❌ 不满足
- 8. 查询本地服务器（最后手段）
 - ❌ 不满足
- 9. 默认 fallback: 128K



9.2 本地服务器类型检测

detect_local_server_type() 通过探测已知端点来识别本地推理服务器：

```
LM Studio → GET /api/v1/models → 检查响应结构
Ollama      → GET /api/tags → 检查 "models" 字段
llama.cpp   → GET /v1/props → 检查 "default_generation_settings"
vLLM        → GET /version → 检查 "version" 字段
```



Note

上下文长度错误的代价极高：Hermes 的 10 级回退链在大部分常见场景下都能找到正确的值，但对于未列出的新模型或冷门 provider，最终会落到 128K fallback，这可能导致 1M 上下文的模型被当作 128K 使用（压缩过旱）。

理想的设计应该提供用户反馈机制：当 fallback 值与实际上下文窗口差异较大时（如 API 返回 context exceeded 错误），应该提示用户设置 model.context_length。

十、设计哲学综合分析

10.1 模块化与可测试性

每个模块 (prompt_builder, context_compressor, context_references, anthropic_adapter, prompt_caching, auxiliary_client) 都是**无状态或纯函数**的，依赖通过参数注入而非全局状态。这使得每个模块都可以独立单元测试，也使得像"切换压缩模型"或"添加新的 @ 引用类型"这样的改动只需要修改对应模块。

10.2 渐进式降级策略

整个系统充满了"如果 X 失败，尝试 Y；如果 Y 也失败，使用 Z"的降级链：

- Provider 解析: OpenRouter → Nous → Custom → Codex → Anthropic → API-key
- 上下文长度: config → 缓存 → 端点元数据 → 硬编码 → 128K fallback
- 压缩摘要: LLM 摘要 → 中间消息直接丢弃（不注入摘要）

最后一种降级（中间消息直接丢弃）是最不理想的，但系统不会因此崩溃。

10.3 安全优先于性能

在安全检查和性能优化的权衡中，Hermes 选择了更安全的方向：

- 上下文文件在注入前经过威胁扫描（即使会消耗少量 CPU）
- @ 引用有严格的 Token 上限（防止上下文淹没）
- 敏感路径被明确拦截（而非默认允许）
- Prompt Caching 的断点数量受 Anthropic 限制（而非尝试突破限制）

Note

安全与性能的根本矛盾：在上下文管理领域，安全检查（扫描、预算控制、路径验证）天然性能敌人——它们需要额外的计算和存储。每次扫描上下文文件会增加系统提示词构建时间；Token 预算检查需要提前估算 content 的 token 数量；@ 引用解析需要检查文件访问权限。

Hermes 的策略是"安全操作在构建时执行一次，运行时不再重复"。这在 CLI 场景（每次请求都是独立的系统提示词构建）中是合理的，但在需要极低延迟的流式场景中，这些安全检查的成本需要重新评估。